MongoDB Relationships — Documentation

1) What is a "Relationship" in MongoDB?

A **relationship** describes how documents in **different (or same) collections** are logically connected. Even though MongoDB is a NoSQL database, you still **model relationships** either by **embedding** related data inside a document or by **referencing** another document's _id and joining with an aggregation $lookup.

The 4 Relationship Types

1. **One-to-One (1:1)** – One document in A relates to **exactly one** document in B.
2. **One-to-Many (1:N)** – One document in A relates to **many** documents in B.
3. **Many-to-One (N:1)** – **Many** documents in A relate to **one** document in B (inverse of 1:N).
4. **Many-to-Many (N:N)** – **Many** documents in A relate to **many** in B.

---

2) Two Ways to Model Relationships

2.1 Embedding (Denormalization)

- Put the related data **inside** the parent document.
- Fewer queries, faster reads for the whole object graph.
- Document can grow large; duplication across documents.

2.2 Referencing (Normalization)

- Store the related document's _**id** and join with $lookup when needed.
- Good for reusability, high-cardinality, large/independent sub-objects.
- Requires extra joins (aggregation), more round trips if not careful.

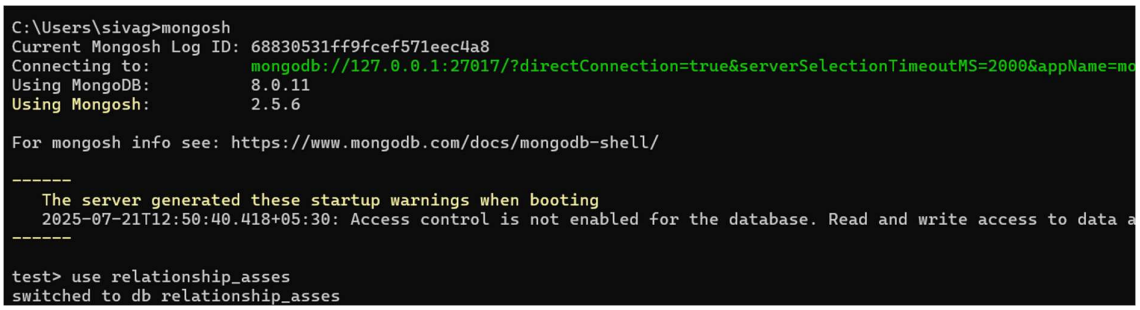
---

3) Create DB & Base Collections (Reference Model)

We'll use **3 collections**: users, posts, tags.

use relationship_asses

📸 **Screenshot:**



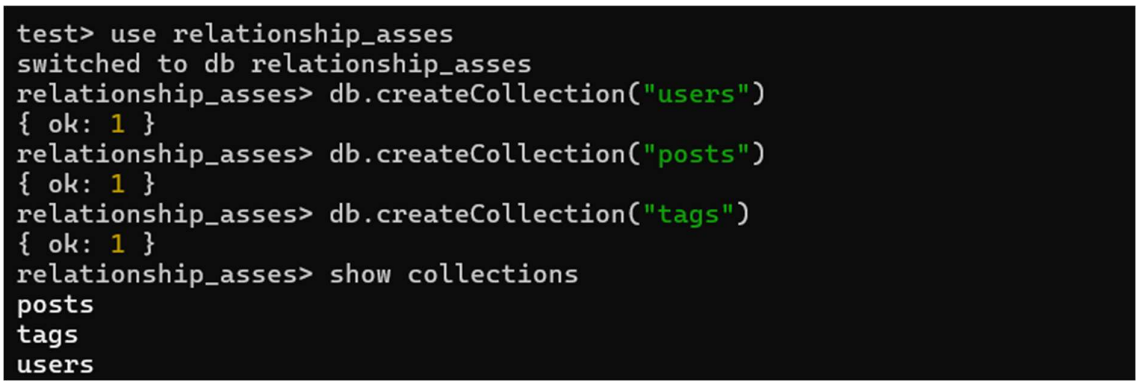
Collection Creation

db.createCollection("users")

db.createCollection("posts")

db.createCollection("tags")

📸 **Screenshot:**



3.1 Insert 5 documents into each (Reference-first design)

users

db.users.insertMany([

{ _id: 1, name: "Alice" },

{ _id: 2, name: "Bob" },

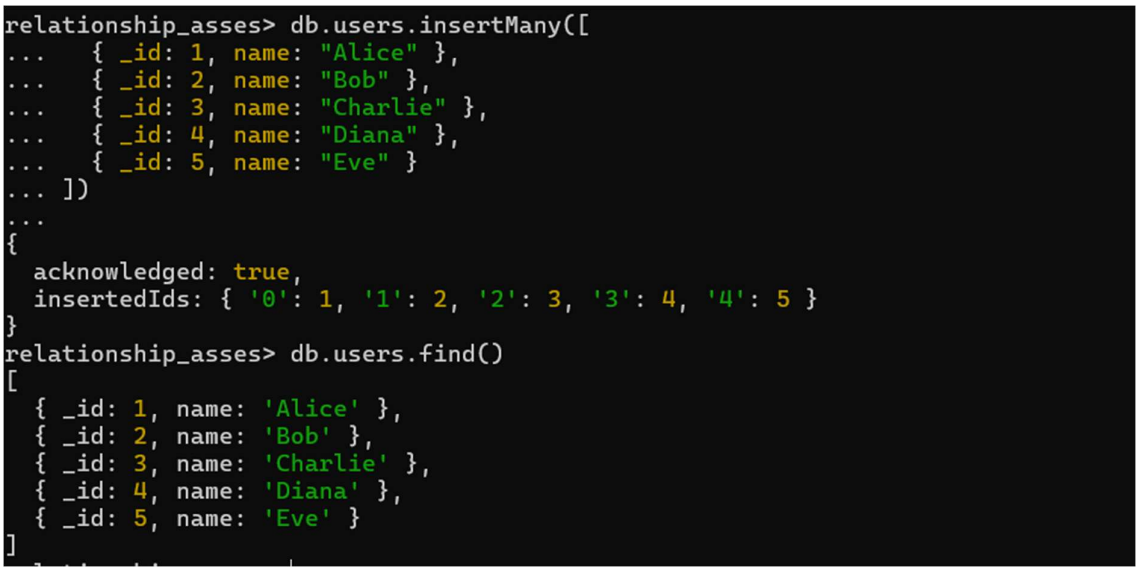
{ _id: 3, name: "Charlie" },

{ _id: 4, name: "Diana" },

{ _id: 5, name: "Eve" }

])

📸 **Screenshot:**

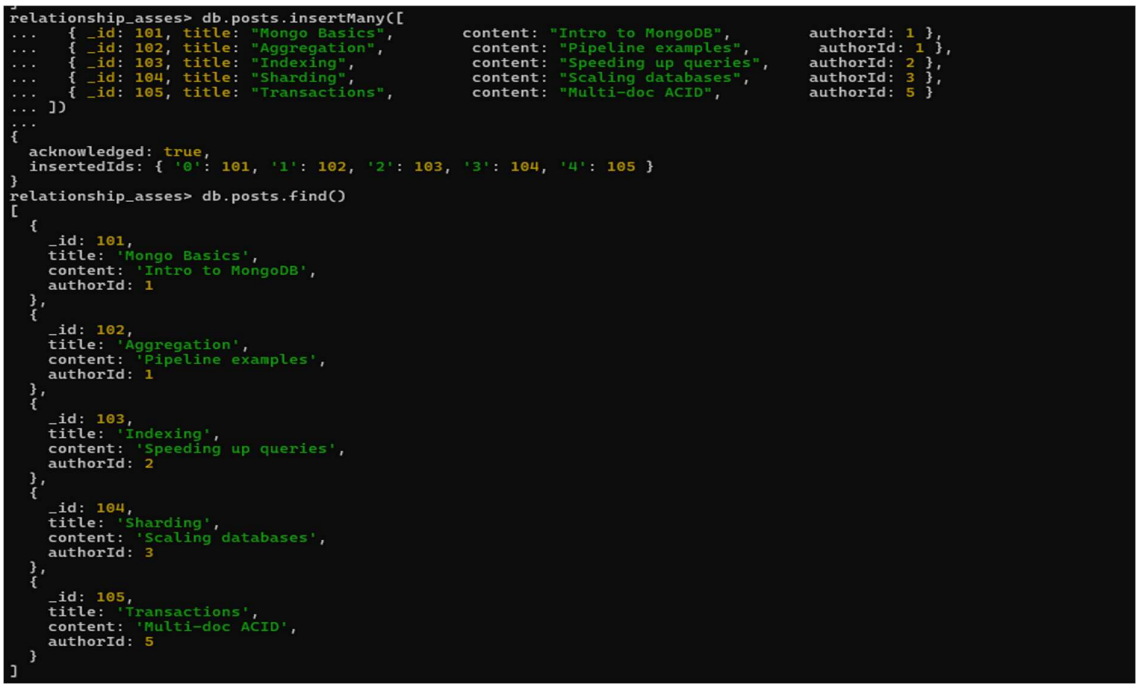


```
relationship_asses> db.users.insertMany([
...   { _id: 1, name: "Alice" },
...   { _id: 2, name: "Bob" },
...   { _id: 3, name: "Charlie" },
...   { _id: 4, name: "Diana" },
...   { _id: 5, name: "Eve" }
... ])
...
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5 }
}
relationship_asses> db.users.find()
[
  { _id: 1, name: 'Alice' },
  { _id: 2, name: 'Bob' },
  { _id: 3, name: 'Charlie' },
  { _id: 4, name: 'Diana' },
  { _id: 5, name: 'Eve' }
]
```

---

posts (each post references its author via authorId)

📸 **Screenshot:**



```
relationship_asses> db.posts.insertMany([
...   { _id: 101, title: "Mongo Basics",       content: "Intro to MongoDB",       authorId: 1 },
...   { _id: 102, title: "Aggregation",         content: "Pipeline examples",      authorId: 1 },
...   { _id: 103, title: "Indexing",           content: "Speeding up queries",    authorId: 2 },
...   { _id: 104, title: "Sharding",           content: "Scaling databases",      authorId: 3 },
...   { _id: 105, title: "Transactions",       content: "Multi-doc ACID",         authorId: 5 }
... ])
...
{
  acknowledged: true,
  insertedIds: { '0': 101, '1': 102, '2': 103, '3': 104, '4': 105 }
}
relationship_asses> db.posts.find()
[
  {
    _id: 101,
    title: 'Mongo Basics',
    content: 'Intro to MongoDB',
    authorId: 1
  },
  {
    _id: 102,
    title: 'Aggregation',
    content: 'Pipeline examples',
    authorId: 1
  },
  {
    _id: 103,
    title: 'Indexing',
    content: 'Speeding up queries',
    authorId: 2
  },
  {
    _id: 104,
    title: 'Sharding',
    content: 'Scaling databases',
    authorId: 3
  },
  {
    _id: 105,
    title: 'Transactions',
    content: 'Multi-doc ACID',
    authorId: 5
  }
]
```

---

tags

db.tags.insertMany([

{ _id: "t1", name: "mongodb" },

{ _id: "t2", name: "scaling" },

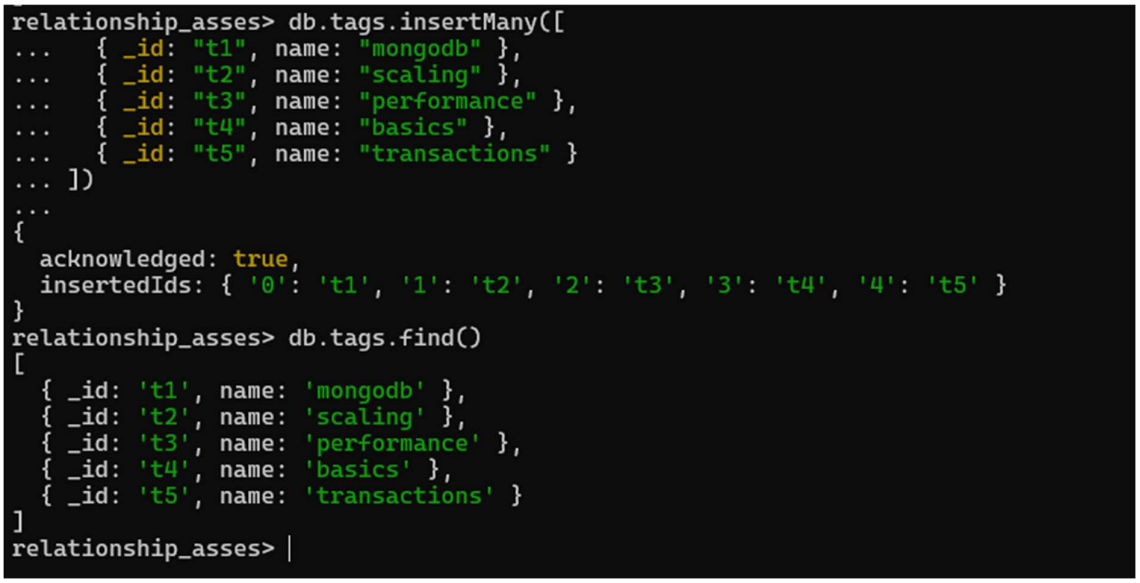
{ _id: "t3", name: "performance" },

{ _id: "t4", name: "basics" },

{ _id: "t5", name: "transactions" }

])

📸 **Screenshot:**



---

4) One-to-One (1:1) — Embedded vs Reference

4.1 Embedded 1:1 — users.profile

Each user has **one profile** stored **inside** the user document.

db.users.updateOne(

{ _id: 1 },

{ $set: { profile: { age: 30, city: "Delhi" } } }

)

📸 **Screenshot**:

db.users.find({_id:1}).pretty()

Explanation:

This query embeds a profile document inside the users document with _id: 1. It represents a **One-to-One (1:1)** relationship using **embedding**. Suitable for small, closely related data like settings or address.

4.2 Referenced 1:1 — profiles collection

Create a separate profiles collection and link by userId.

db.createCollection("profiles")

db.profiles.insertMany([

{ _id: 1001, userId: 1, fullName: "Alice Johnson", bio: "Full-stack dev" },

{ _id: 1002, userId: 2, fullName: "Bob Martin", bio: "Cloud architect" }

])

Join (user + profile) using $lookup

📸 **Screenshot**:

```
relationship_asses> db.users.aggregate([
...   {
...     $lookup: {
...       from: "profiles",
...       localField: "_id",
...       foreignField: "userId",
...       as: "profile"
...     }
...   },
...   { $unwind: "$profile" },
...   { $project: { name: 1, "profile.fullName": 1, "profile.bio": 1 } }
... ])
...
[
  {
    _id: 1,
    name: 'Alice',
    profile: { fullName: 'Alice Johnson', bio: 'Full-stack dev' }
  },
  {
    _id: 2,
    name: 'Bob',
    profile: { fullName: 'Bob Martin', bio: 'Cloud architect' }
  }
]
```

Explanation:

This aggregation query joins the users collection with the profiles collection using userId. It models a **One-to-One** relationship using **referencing**. Ideal when the related data has its own lifecycle.

---

5) One-to-Many (1:N) — Embedded vs Reference

5.1 Embedded 1:N — users.posts[]

Put a **small subset** of each post inside the user.

db.users.updateOne(

{ _id: 1 },

{ $set: {

posts: [

{ id: 201, title: "My Embedded Post 1" },

{ id: 202, title: "My Embedded Post 2" }]}})

Query embedded posts of Alice:

db.users.find({ _id: 1 }, { name: 1, posts: 1, _id: 0 })

📸 **Screenshot**:

```
relationship_asses> db.users.find({ _id: 1 }, { name: 1, posts: 1, _id: 0 })
[
  {
    name: 'Alice',
    posts: [
      { id: 201, title: 'My Embedded Post 1' },
      { id: 202, title: 'My Embedded Post 2' }
    ]
  }
]
relationship_asses>
```

Explanation:

This query embeds a small array of posts inside the user document. It is a **One-to-Many (1:N)** relationship using **embedding**. Best when posts are few and always needed with the parent.

5.2 Referenced 1:N — users → posts

We already inserted posts with authorId. Join them:

📸 **Screenshot**:

```
relationship_asses> db.posts.aggregate([
...   {
...     $lookup: {
...       from: "users",
...       localField: "authorId",
...       foreignField: "_id",
...       as: "author"
...     }
...   },
...   { $unwind: "$author" },
...   { $project: { title: 1, author: "$author.name" } }
... ])
...
[
  { _id: 101, title: 'Mongo Basics', author: 'Alice' },
  { _id: 102, title: 'Aggregation', author: 'Alice' },
  { _id: 103, title: 'Indexing', author: 'Bob' },
  { _id: 104, title: 'Sharding', author: 'Charlie' },
  { _id: 105, title: 'Transactions', author: 'Eve' }
]
```

Explanation:

This is a **One-to-Many (1:N)** relationship modeled using **referencing**. Each post references its author by authorId, and the $lookup fetches the author's details. It's efficient for large child sets.

---

6) Many-to-One (N:1) — Embedded vs Reference

N:1 is simply the **inverse** view of 1:N. We'll show how to query it both ways.

6.1 Embedded N:1 — Find the parent document of an embedded child

Which user owns an embedded post with id: 201?

📸 **Screenshot:**

```
relationship_asses> db.users.find(
...   { "posts.id": 201 },
...   { name: 1, "posts.$": 1 }
... )
...
[
  {
    _id: 1,
    name: 'Alice',
    posts: [ { id: 201, title: 'My Embedded Post 1' } ]
  }
]
```

Explanation:

This simple query fetches all posts written by the user with _id: 1. This is a **Many-to-One (N:1)** relationship — many posts belong to one user.

6.2 Referenced N:1 — Many posts → one user

List all posts of authorId = 1 (Alice):

db.posts.find({ authorId: 1 }, { _id: 0, title: 1 })

📸 **Screenshot:**

```
relationship_asses> db.posts.find({ authorId: 1 }, { _id: 0, title: 1 })
[ { title: 'Mongo Basics' }, { title: 'Aggregation' } ]
relationship_asses>
```

Explanation:

This Fetches all post titles written by user with authorId: 1, excluding _id. Represents a many-to-one relationship from posts to a single user.

---

7) Many-to-Many (N:N) — Embedded vs Reference

7.1 Embedded N:N

Find posts that contain tag "mongodb"

📸 **Screenshot:**

```
relationship_asses> db.posts.find({ tagNames: "mongodb" }, { title: 1, tagNames: 1 })
[
  {
    _id: 101,
    title: 'Mongo Basics',
    tagNames: [ 'mongodb', 'basics' ]
  }
]
```

Explanation:

This query retrieves posts where "mongodb" exists in the embedded tagNames array. Displays title and tags. Demonstrates many-to-many using embedded tag values.

7.2 Referenced N:N

Join posts with tags:

```
relationship_asses> db.posts.aggregate([
...   {
...     $lookup: {
...       from: "tags",
...       localField: "tagIds",
...       foreignField: "_id",
...       as: "tags"
...     }
...   },
...   { $project: { title: 1, tagNames: "$tags.name" } }
... ])
[
  {
    _id: 101,
    title: 'Mongo Basics',
    tagNames: [ 'mongodb', 'basics' ]
  },
  {
    _id: 102,
    title: 'Aggregation',
    tagNames: [ 'mongodb', 'performance' ]
  },
  { _id: 103, title: 'Indexing', tagNames: [ 'performance' ] },
  { _id: 104, title: 'Sharding', tagNames: [ 'scaling' ] },
  { _id: 105, title: 'Transactions', tagNames: [ 'transactions' ] }
]
relationship_asses>
```

Explanation:

This query joins posts with tags using tag IDs. Fetches each post's title along with matching tag names. Demonstrates a **many-to-many** relationship using **referencing** and $lookup for linked data.

📌 Conclusion

This documentation provides a complete hands-on guide for understanding and implementing MongoDB relationships. Here's what I've done:

- ✅ Explained **what relationships are** in MongoDB and the **4 key types**: One-to-One, One-to-Many, Many-to-One, Many-to-Many.
- ✅ Learned **two modeling techniques**:
  - **Embedding**: When data is tightly coupled, small, and always needed together.
  - **Referencing**: When data is large, shared, or accessed independently.
- ✅ Created a MongoDB database with 3 collections (users, posts, tags) and inserted sample data.
- ✅ Demonstrated all 4 relationships in **both embedded and reference forms**, with clear **MongoDB queries and results**.
- ✅ Provided **query explanations** to show what each operation does and **when to use it**.

📑 When to Use What

Use Embedding When...	Use Referencing When...
Data is always retrieved together	Data is accessed independently
Relationship is one-to-one or one-to-few	Relationship is one-to-many or many-to-many

---